CounselCopilot

Catching Policy and Legal Conflicts in NDA Drafts with Applicability-Aware Retrieval and Verified Speculation

LMA Group Project 2026 – Project Proposal

Team members (roll numbers):

2025201049 2025201044 2025201068 2025201038

Summary in three lines

Problem: legal consultants who draft NDAs change often and cannot learn hundreds of pages of company policy, so their drafts quietly *miss* required terms or break rules.
Idea: a drafting copilot that finds every company rule and Indian-law provision that *applies to this deal*, checks the draft against each one with exact tools, and cites both sides. While the consultant is still writing, it runs the review early and reuses the result only if the finished clause matches.
Test: on real NDAs with expert-labelled and planted defects, does this catch more omissions with fewer false alarms than standard RAG, long-context prompting and checking every rule, and does early review cut waiting time without changing the result?

1 Domain, target user and problem

Domain: Legal. **Scope:** one contract type (NDAs), one jurisdiction (India), three companies' policy packs.
Target user: a legal consultant (contract staff, outsourced counsel or junior associate) drafting NDAs for a company, with a senior lawyer who signs off.

Problem statement. Companies spread their NDA requirements across many documents: data-protection, IP and vendor policies, an approval matrix and past negotiated positions. The consultants who draft NDAs rotate often, and a new consultant cannot absorb all of this in a few days. Their typical mistake is not bad wording but a *missing* or non-compliant term, such as no data-return clause, a confidentiality period shorter than policy allows, or a foreign governing law that needs General Counsel approval. Some drafts also contain terms that are risky under Indian law, such as a broad non-compete, which is void as a restraint of trade under s.27 of the Indian Contract Act, 1872. These misses surface later as disputes. A reviewer that reads the draft once cannot find a missing clause, because there is nothing in the text to point at. We want a copilot that checks every applicable rule, cites it, and acts as knowledge transfer at the moment the consultant needs it.

Why it matters. NDAs are the most frequently signed contracts, consultant turnover is high, and one missed rule can expose a company's data or IP. A senior lawyer can verify a cited flag in seconds but cannot re-read every policy for every draft.

2 What is missing in existing approaches

- **Contract-error benchmarks** (CLAUSE, EACL Findings 2026; ContractScrub, 2026) test single prompted models on errors inside one contract; the best macro recall on ContractScrub is only 0.75. **ContractEval** (NLLP 2025) finds that open models wrongly answer "no such clause" in about 25–30% of cases. None of them checks a draft against a *company's own policies*.
- **PolicyGuard** (arXiv 2606.32004, 2026), the closest work, compiles one company's NDA playbook into logic rules and checks every rule. It is evaluated on 5 confidential contracts that cannot be released, treats absence only through per-rule questions, and ignores deal-specific applicability.
- **Industry tools** (Harvey, LegalOn, Spellbook) offer "playbook review" with private evaluations only. **There is no open benchmark for policy compliance of NDAs that tests omissions.**
- **Speculative agents** (Speculative Actions, PASTE; Speculative Interaction Agents, 2026) start work early to hide latency. The last of these speculates on unfinished user speech but loses a little accuracy. None of them verifies reuse exactly or studies document drafting.

3 Research questions and hypotheses

RQ1 (main). Does *applicability-aware two-way retrieval* catch more policy omissions and violations, with fewer false alarms, than (a) standard clause-to-policy RAG, (b) putting the whole policy pack into a long-context model, and (c) checking every rule exhaustively?

H1. It raises omission recall by **≥15 points** over forward-only RAG at equal or fewer false alarms per NDA. It matches exhaustive checking within 3 points of recall while making **≥50% fewer checks**, and its advantage over long context **grows with policy size**.

RQ2. Does *speculative clause review* cut the consultant's time-to-feedback without changing the result?

H2. Median time-to-feedback per clause drops by **≥40%** in live replay, and **≥98%** of reused verdicts are identical to a full re-run.

4 Proposed system

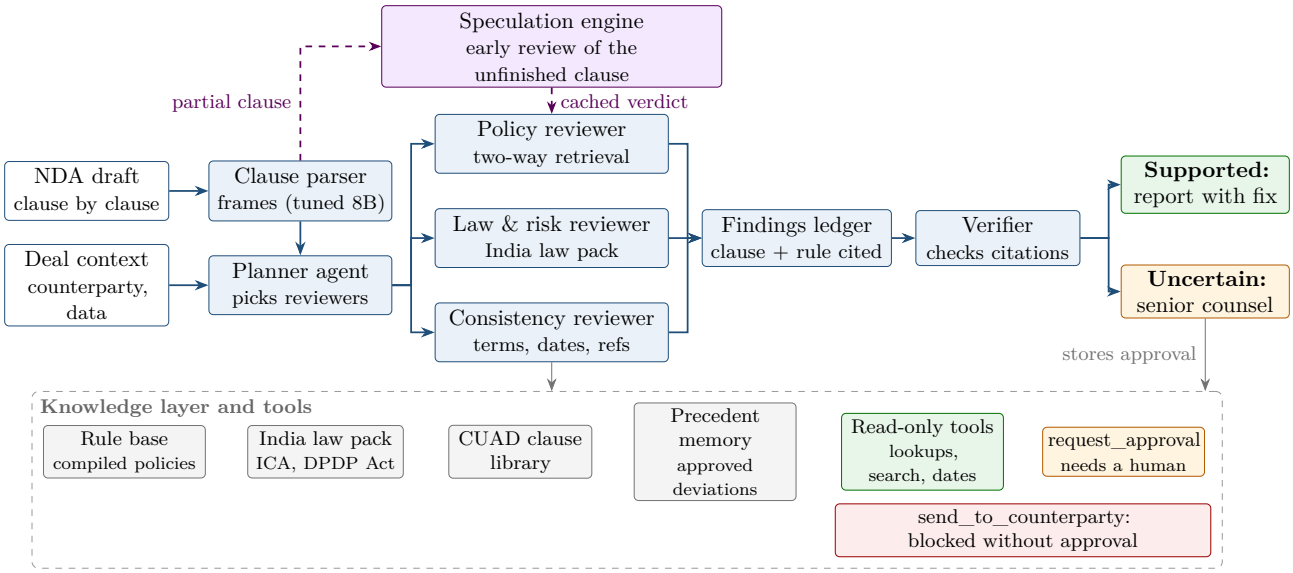


Figure 1: CounselCopilot architecture. Blue boxes are LLM agents (same backbone in every system we compare). Green tools are read-only and may be called speculatively; orange needs human approval; red is never allowed without approval.

How it works. The *clause parser* turns each clause into an *obligation frame* {party, action, object, condition, duration, exception}, so numbers and conditions can be compared exactly rather than by text similarity. Offline, a *policy compiler* turns each company's prose policies into a rule base, where every rule has an ID, an applicability condition (e.g. `vendor AND personal_data`), a requirement and its source text. The *planner* picks reviewers based on the deal and the clause types present. Three reviewers call tools and write findings to a shared ledger. The *verifier* keeps a finding only if both citations (NDA clause and rule or statute) support it, and sends uncertain or approval-gated cases to a senior lawyer. Approved deviations go into *precedent memory*, so the same question is not escalated twice.

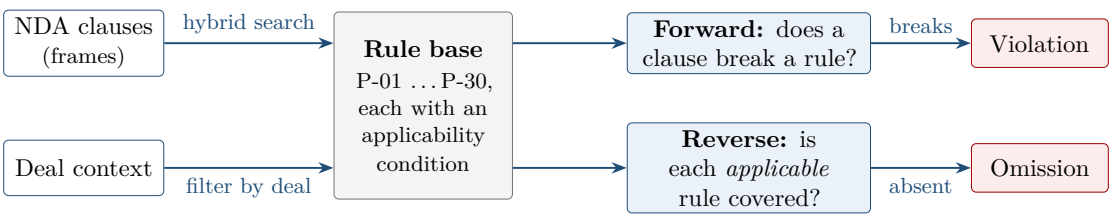


Figure 2: Two-way retrieval. The reverse path is the only way to find a *missing* clause, and the deal filter stops rules that don't apply to this deal from firing.

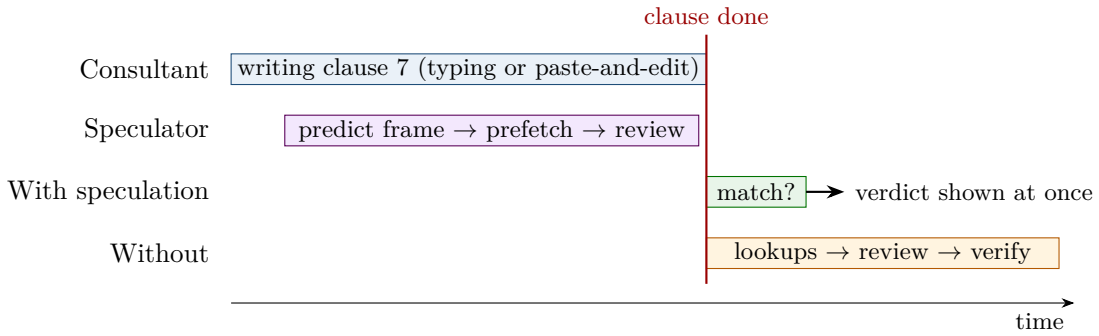


Figure 3: Speculative clause review. The cached verdict is shown only if the finished clause's frame matches the predicted one on every field the rules test. Otherwise the review runs normally, so the output never changes.

RAG design. Retrieval combines BM25 and dense search with a cross-encoder reranker, so legal terms like "indemnify" match exactly. Policies are chunked by rule rather than by fixed size, so an exception stays with its rule. A one-hop cross-reference graph pulls in linked approval rules and statutory exceptions. Because Gemini can read a whole 40k-token pack, we do not assume RAG is needed. We *measure* it with a policy-size sweep at 40k, 150k and 400k tokens.

Speculation, and its safety boundary. Only read-only tools and the review computation itself are ever run early. `request_approval` and `send_to_counterparty` are never speculated, and a speculative verdict never triggers an action. Speculative RAG (a small drafter writes one finding per evidence source, then the backbone verifies them) is our stretch goal, tested for accuracy.

5 The five capabilities: why each is needed and how we measure it

Capability	Why this problem needs it	How we measure it
Fine-tuning	One QLoRA Llama-3.1-8B for clause tags (CUAD), ContractNLI checks, obligation frames and drafts. Frames must be exactly structured, and frame prediction runs after every sentence, so it must be local and free.	CUAD tag F1; ContractNLI acc./macro-F1 (12 trained, 5 held-out checks); frame field accuracy on 300 gold frames; tuned vs untuned vs prompted Gemini
RAG	An omission check must find *every* applicable rule, cite it exactly and stay cheap as the policy corpus grows.	Applicable-rule recall@k, omission recall, nDCG@10, policy-size sweep vs long context
Reasoning	Deciding whether a rule applies to this deal, comparing frames to rules, verifying each finding.	Accuracy of every intermediate applicability decision vs gold; verifier rejection precision; cause of each miss
Agentic	Planner, specialist reviewers, shared ledger, precedent memory, hand-off to a senior lawyer.	Planner vs fixed pipeline; hand-off precision/recall; escalations avoided by memory
Tool calling	Exact bookkeeping (terms, dates, cross-references), rule/statute/approval lookups, gated actions, prefetching.	Tool precision/recall vs gold tool sets per defect type, argument validity, recovery from 10% injected errors, **unsafe action rate (target 0)**

6 Dataset and evaluation environment: PolicyNDA

- **NDAs:** ContractNLI (607 real NDAs, split 423/61/123, CC BY 4.0). **Clause library:** CUAD (510 contracts, 41 clause types).
- **Company policy packs (we write):** 3 fictional Indian companies, each with ∼30 NDA rules buried in ∼40k tokens of realistic prose. They are written by teammates who do not build the system, and the gold rule lists are kept separate.
- **India law pack:** ∼15 clear-cut provisions (ICA s.23/27/28, DPDP Act 2023, Arbitration Act seat rules, Specific Relief Act), each checked by two teammates.
- **Ground truth without circularity:** about 60% of the rules are anchored to ContractNLI's

17 expert-labelled checks (e.g. "vendor NDAs must require return of information" ↔ *return of confidential information*). So each real NDA's *natural* omissions and violations are already labelled by legal experts. The remaining field rules (governing law, term length) are hand-labelled by two annotators, with Cohen's κ reported.

- **Planted defects on top:** violations, omissions, approval-gated deviations, statutory risks, internal inconsistencies, prompt-injection text, and *distractors* (rules that look relevant but don't apply), giving 2–4 per NDA. About 25% of NDAs get only distractors, so we can measure false alarms.
- **Live replay:** each test NDA is replayed as a drafting session in two styles, typed at 40 wpm and paste-a-template-then-edit.

7 Evaluation plan

Two-stage backbone. Stage 1 runs everything on **Llama-3.1-8B** served free on Kaggle GPUs. Stage 2 confirms the main systems on **Gemini 3.8 Flash**. All systems in a comparison use the same backbone. Long-context runs and the policy-size sweep need Gemini, because a T4 GPU cannot hold a 40k-token prompt.

Baselines	B1 long context (whole pack in one prompt) · B2 naive RAG (fixed chunks, forward only) · B3 single agent with tools · B4 exhaustive per-rule checking (PolicyGuard-style)
Ablations (one change each)	no reverse retrieval (clean RQ1 test) · no deal filter · no frames · no verifier · planner → fixed pipeline · untuned model · no cross-reference graph · no memory · speculation: none / prefetch only / heuristic frames / tuned frames / oracle
Detection	Recall, precision and F1 per defect type; **omission recall**; false alarms per NDA. A finding counts only if both citations are correct. Natural and planted defects are reported separately.
Speculation	Time-to-feedback p50/p95 (real measured latency, including GPU contention); reuse rate; agreement of reused verdicts with a full re-run; wasted calls and cost
Safety	Unsafe/unapproved action rate (target 0), including prompt-injection text inside NDAs; hand-off precision and recall
Rigour	Bootstrap 95% CIs and McNemar's test. An LLM judge is used only for explanation quality, validated against 100 human labels (Cohen's κ). Five worked qualitative cases and a table of misses by cause.

8 Expected novelty

1. **PolicyNDA**, the first *open* benchmark for checking NDAs against company policy. It covers omissions and deal-specific applicability, with ground truth anchored to expert labels.
2. **Applicability-aware two-way retrieval**, compared honestly against exhaustive per-rule checking and against long context as the policy corpus grows.
3. **Verified speculative review** for live drafting: reuse is exact rather than approximate, with a strict read-only boundary.

9 Safety plan

- Only public NDAs and synthetic policies, so no client or personal data. Llama runs locally; only public or synthetic text goes to the Gemini API, on the paid tier.
- Gated actions: `request_approval` needs human confirmation, and `send_to_counterparty` is a mock that is blocked without approval. Misuse is measured, including under prompt injection.
- The output is decision support with citations, not legal advice. No team member is a lawyer, so statutory flags are worded as "risk under s.27" and the law pack is small, cited and dated.

10 Compute plan

Item	Resource	Estimate
QLoRA Llama-3.1-8B, multi-task, ∼18k examples, 2 runs	Kaggle T4 (free)	≈16 GPU-h
Tuned-model inference (frames for replay, ContractNLI test)	Kaggle T4, vLLM	≈6 GPU-h
Stage 1: all systems and ablations on Llama-8B, 123 NDAs, 2 seeds	Kaggle 2×T4, vLLM	≈30 GPU-h, $0
Stage 2: B1–B4, full system and 6 core ablations, 123 NDAs (≈68M in / 7M out tokens)	Gemini 3.8 Flash, Batch API	≈$38
Policy-size sweep, live-latency replay on Gemini	Gemini 3.8 Flash	≈$35
Silver frames, policy compilation, Pro reference runs	Gemini 3.1 Pro, Batch API	≈$28
Total		**≈55 GPU-h, ≈$100**

Prices from the official Gemini pricing page (2 Oct 2026): 3.8 Flash costs $0.75 in / $3.75 out per million tokens until 31 Dec 2026, then double, so the API total is ≈$175 if stage 2 runs after that date. 55 GPU-hours is about two weeks of Kaggle's free quota.

11 Risks and fallback plan

Risk	Fallback
Scope too large for the time	RQ1 is the core. Speculative RAG is dropped first; speculative review is cheap once frames exist.
Long context or exhaustive checking matches our retrieval	Report it. The benchmark, the omission results and the cost difference are still contributions.
Llama-8B too weak for tool use	Constrained JSON decoding; move stage 1 to a hosted Llama-70B; run stage 2 regardless.
Gains come from memorised ContractNLI labels	5 checks are held out of fine-tuning and reported separately.
Paste-and-edit leaves little time to speculate	Report reuse rate against how much of the clause is written; this is a finding, not a failure.
Unrealistic policies or law errors	Model policies on public corporate policies; realism check by a teammate; two-person law check.

Key references. Koreeda & Manning, ContractNLI (EMNLP Findings 2021) · Hendrycks et al., CUAD (NeurIPS 2021) · CLAUSE (EACL Findings 2026, arXiv:2511.00340) · ContractScrub (arXiv:2608.20204) · ContractEval (NLLP 2025, arXiv:2508.03080) · PolicyGuard (arXiv:2606.32004) · Speculative Interaction Agents (arXiv:2605.13360) · Speculative Actions (ICLR 2026) · Speculative RAG (ICLR 2025) · Lost in the Middle (TACL 2024) · QLoRA (NeurIPS 2023).